

Vite & Gourmand

TRAITEUR ARTISANAL · BORDEAUX

Documentation technique

Architecture, modélisation des données, API REST et sécurité
de la plateforme Vite & Gourmand.

Version 1.0

· July 2026

Sommaire

1. Présentation générale	03
1.1 Contexte et objectifs	03
1.2 Stack technique retenue	03
1.3 Choix d'architecture	04
2. Architecture applicative	05
2.1 Vue d'ensemble	05
2.2 Découpage en couches	06
2.3 Choix de la double BDD (SQL + NoSQL)	06
3. Modélisation des données	07
3.1 Modèle Conceptuel (MCD)	07
3.2 Modèle Logique (MLD)	08
3.3 Dictionnaire de données	09
3.4 Modélisation NoSQL (MongoDB)	13
4. Diagrammes UML	14
4.1 Diagramme de cas d'utilisation	14
4.2 Diagramme de séquence — Création de commande	15
5. API REST	16
5.1 Conventions générales	16
5.2 Liste complète des endpoints	16
5.3 Codes de réponse HTTP	20
6. Sécurité	21
6.1 Authentification et session	21
6.2 Contrôle d'accès (RBAC)	21
6.3 Rate limiting et anti-abus	22
6.4 Conformité RGPD	22

7. Règles métier	23
7.1 Calcul des prix et réduction	23
7.2 Cycle de vie des commandes	23
7.3 Modération des avis	24

1. Présentation générale

1.1 Contexte et objectifs

Vite & Gourmand est une plateforme web full-stack conçue pour permettre à un traiteur artisanal bordelais de gérer l'ensemble de son activité en ligne : présentation de son catalogue de menus, prise de commande par des clients particuliers ou professionnels, gestion opérationnelle du flux des commandes par les employés, et pilotage stratégique par l'administrateur.

Le projet est développé dans le cadre de l'Épreuve de Certification Finale du Titre Professionnel *Développeur Web et Web Mobile* (Studi). Il satisfait aux exigences fonctionnelles du cahier des charges tout en respectant les bonnes pratiques modernes de développement : séparation des responsabilités, sécurité, testabilité, accessibilité.

1.2 Stack technique retenue

Couche	Technologie	Version	Justification
Front-end	HTML5 + Tailwind CSS + Alpine.js	Tailwind 3, Alpine 3	Approche pragmatique : pas de build lourd, page-par-page, dégradation gracieuse.
Back-end	PHP 8.2+ · Symfony 7	Symfony 7.x	Framework mature, écosystème riche (Security, Validator, Mailer), typage strict PHP 8.
ORM	Doctrine ORM	3.x	Intégration Symfony native, migrations versionnées, contraintes en base et applicatives.
BDD relationnelle	PostgreSQL	15+	Contraintes CHECK évoluées, JSON natif, performance sur les jointures.
BDD NoSQL	MongoDB	6+	Statistiques dénormalisées (exigence explicite du sujet).
Authentification	Sessions PHP + cookies HttpOnly	—	Simplicité, sûreté (pas de JWT stockage local, résistance au XSS).
Hashage	Argon2id	—	Recommandation ANSSI / OWASP pour les mots de passe.
Templating email	Twig	3.x	Standard Symfony, séparation logique / présentation.

1.3 Choix d'architecture

Une API REST monolithique modulaire

Nous avons volontairement rejeté une architecture microservices pour la version 1.0, jugée disproportionnée pour la volumétrie attendue (quelques centaines de commandes par mois). Le back-end est un monolithe modulaire découpé en **sept domaines métier** (Auth, Menus & Plats, Commandes, Admin, Avis, Contact, Horaires), chacun disposant de ses propres entités, repositories, services et contrôleurs. Cette organisation prépare une éventuelle bascule vers des services séparés si le trafic le justifie plus tard.

Front / back découplés

Le front-end est une SPA légère (pas de framework lourd type React ou Vue) qui communique avec le back-end via une API REST JSON. Ce choix permet de conserver un front simple (HTML + Tailwind + Alpine), tout en respectant strictement le principe de responsabilité : le back-end n'a aucune connaissance des vues, il expose uniquement des données.

Base NoSQL pour les statistiques

Le cahier des charges impose l'usage d'une base NoSQL. Nous avons retenu MongoDB pour stocker une projection dénormalisée des commandes, exclusivement dédiée aux agrégations statistiques du tableau de bord administrateur. PostgreSQL reste la source unique de vérité : MongoDB peut être resynchronisée à volonté via une commande CLI dédiée (`php bin/console app:sync-mongo`).

2. Architecture applicative

2.1 Vue d'ensemble

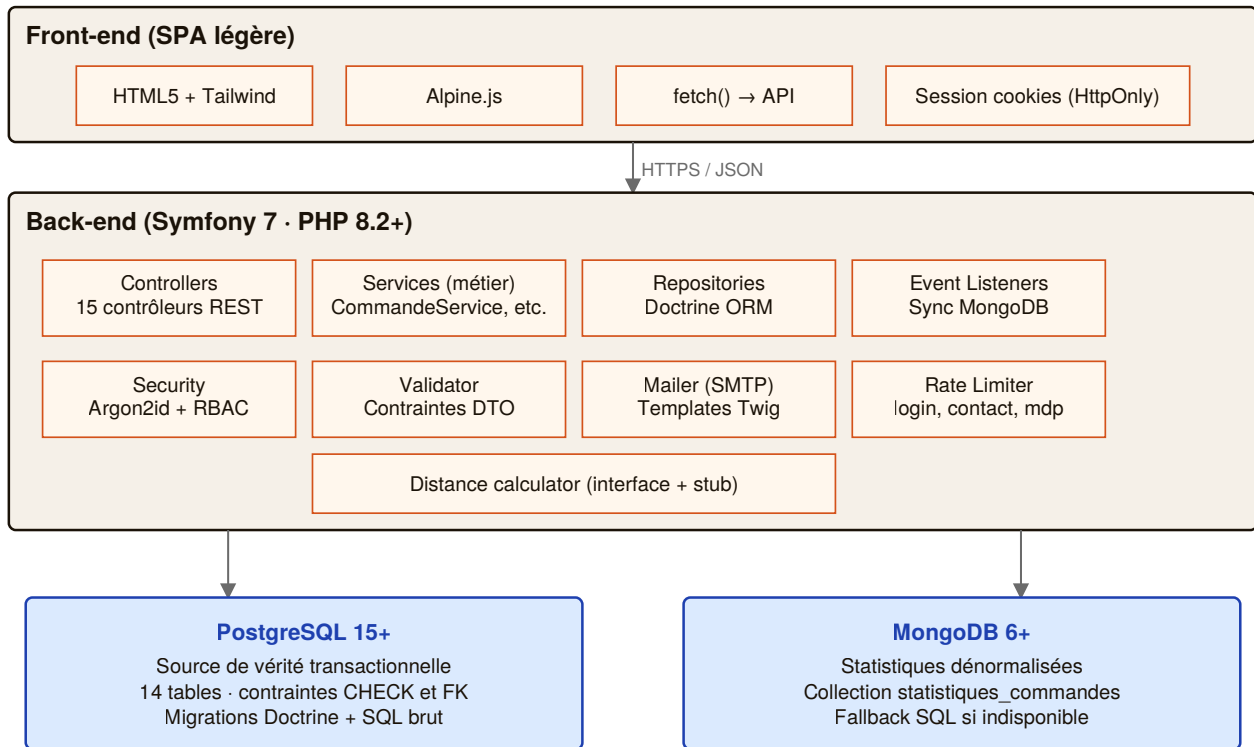


Schéma d'architecture — Front → API REST → couche métier → double BDD

2.2 Découpage en couches

Le back-end respecte une architecture en trois couches strictes :

Couche Contrôleur (adaptateur HTTP)

Les 15 contrôleurs se limitent à quatre responsabilités : (1) désérialiser le corps HTTP en DTO, (2) valider les DTO via le `ValidatorInterface`, (3) déléguer au service métier approprié, (4) sérialiser la réponse et gérer les codes HTTP. Aucune règle métier ne figure dans les contrôleurs — cela garantit leur simplicité et leur testabilité.

Couche Service (métier)

Les 15 services encapsulent toute la logique métier : calcul des prix, transitions d'état, génération de numéros uniques, envois d'e-mails transactionnels, synchronisation MongoDB. Chaque service est *injectable* et *stateless*, ce qui facilite les tests unitaires (mocking des repositories).

Couche Repository (accès données)

Les 12 repositories Doctrine encapsulent toutes les requêtes SQL. Aucun contrôleur ni service ne construit de `QueryBuilder` directement — cela garantit que la couche de persistance reste échangeable (par exemple pour un passage à MySQL).

2.3 Choix de la double BDD (SQL + NoSQL)

Deux bases de données sont utilisées, avec des rôles nettement distincts :

Aspect	PostgreSQL	MongoDB
Rôle	Source de vérité transactionnelle	Projection dénormalisée pour statistiques
Données	14 tables normalisées, contraintes CHECK & FK	1 collection <code>statistiques_commandes</code>
Alimentation	Toutes les écritures applicatives	Automatique via listener Doctrine <code>postPersist</code> / <code>postUpdate</code>
Lecture	Toutes les vues métier	Uniquement le dashboard admin
Disponibilité	Critique (bloque l'API si down)	Non critique (fallback SQL transparent)

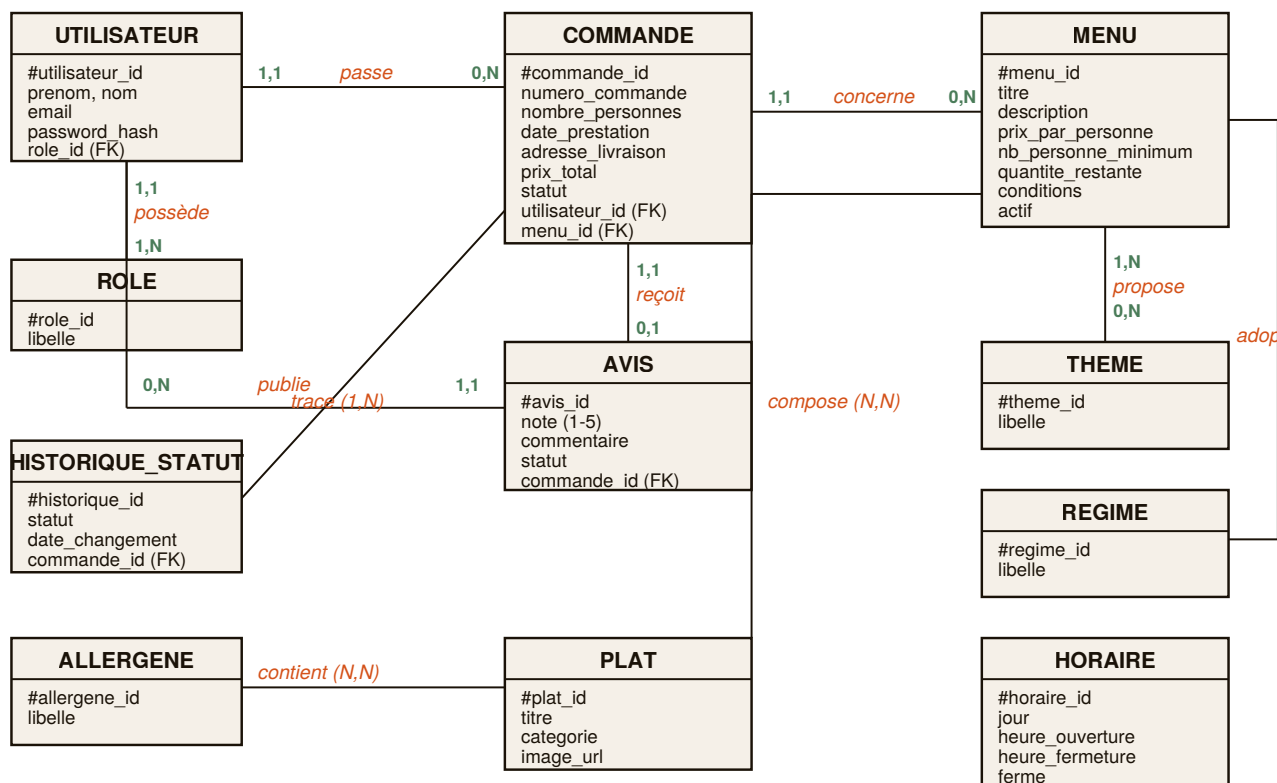
JUSTIFICATION DU REPLI

Le `StatistiquesService` teste la disponibilité de MongoDB avant chaque lecture. Si la base est injoignable, une agrégation Doctrine équivalente est exécutée sur PostgreSQL. Le résultat est identique côté client — seuls les temps de réponse diffèrent. Cette stratégie *fail-safe* respecte l'invariant : « *une panne NoSQL ne doit jamais rendre l'API indisponible* ».

3. Modélisation des données

3.1 Modèle Conceptuel de Données (MCD)

Le MCD ci-dessous représente les entités du domaine et leurs relations conformément au sujet initial. Les cardinalités sont indiquées selon la notation Merise (*min*, *max*).



Modèle Conceptuel de Données — 11 entités, 8 associations

3.2 Modèle Logique de Données (MLD)

Le passage du MCD au MLD suit les règles classiques : les associations plusieurs-à-plusieurs sont matérialisées par des tables pivot ; les associations un-à-plusieurs deviennent des clés étrangères. Les tables physiques créées sont les suivantes :

Table	Rôle	Clé étrangère(s)
<code>utilisateur</code>	Comptes clients, employés, admin	<code>role_id</code>
<code>role</code>	Référentiel des rôles	—
<code>menu</code>	Catalogue des menus	—
<code>plat</code>	Plats individuels	—

theme	Thèmes (Noël, Pâques...)	—
regime	Régimes alimentaires	—
allergene	Allergènes	—
image_menu	Galerie d'images d'un menu	menu_id
menu_theme	Pivot menu ↔ thème	menu_id , theme_id
menu_regime	Pivot menu ↔ régime	menu_id , regime_id
menu_plat	Pivot menu ↔ plat	menu_id , plat_id
plat_allergene	Pivot plat ↔ allergène	plat_id , allergene_id
commande	Commandes clients	utilisateur_id , menu_id
historique_statut_commande	Timeline des statuts	commande_id
avis	Avis clients	utilisateur_id , commande_id , moderateur_id
horaire	Horaires d'ouverture	—

3.3 Dictionnaire de données

Chaque colonne des tables principales est décrite ci-dessous. Les tables pivot et référentielles simples sont omises pour la lisibilité (leur structure est triviale).

Table utilisateur

Colonne	Type	Contraintes	Rôle
utilisateur_id	SERIAL	PK	Identifiant technique
prenom	VARCHAR(100)	NOT NULL	Prénom
nom	VARCHAR(100)	NOT NULL	Nom de famille
email	VARCHAR(180)	NOT NULL, UNIQUE	Identifiant de connexion
password	VARCHAR(255)	NOT NULL	Hash Argon2id
telephone	VARCHAR(20)	NOT NULL	Contact GSM
adresse_postale	TEXT	NOT NULL	Adresse par défaut
role_id	INTEGER	FK role	Rôle attribué
actif	BOOLEAN	DEFAULT TRUE	Soft delete
date_creation	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Inscription
token_reset_password	VARCHAR(64)	NULL, hash SHA-256	Réinitialisation mdp
token_reset_expire_at	TIMESTAMP	NULL	Expiration du token (60 min)

Table **menu**

Colonne	Type	Contraintes	Rôle
menu_id	SERIAL	PK	Identifiant
titre	VARCHAR(150)	NOT NULL	Nom commercial
description	TEXT	NOT NULL	Descriptif complet
prix_par_personne	DECIMAL(8,2)	> 0 (CHECK)	Prix unitaire
nombre_personne_minimum	SMALLINT	> 0 (CHECK)	Seuil de commande
quantite_restante	INTEGER	NULL ou ≥ 0	NULL = illimité
conditions	TEXT	NULL	Conditions particulières
actif	BOOLEAN	DEFAULT TRUE	Soft delete
date_creation	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	—
date_modification	TIMESTAMP	NULL	Auto-remplie

Table **commande**

Colonne	Type	Contraintes	Rôle
commande_id	SERIAL	PK	Identifiant technique
numero_commande	VARCHAR(20)	NOT NULL, UNIQUE	Format VG-YYYY-NNNN
utilisateur_id	INTEGER	FK, ON DELETE RESTRICT	Client
menu_id	INTEGER	FK, ON DELETE RESTRICT	Menu commandé
nombre_personnes	SMALLINT	> 0 (CHECK)	Convives
date_prestation	DATE	NOT NULL	Jour du service
heure_livraison	TIME	NOT NULL	Heure attendue
adresse_livraison	VARCHAR(255)	NOT NULL	—
ville_livraison	VARCHAR(100)	NOT NULL	Test Bordeaux/hors
distance_km	DECIMAL(6,2)	≥ 0 (CHECK)	Depuis Bordeaux
prix_menu	DECIMAL(10,2)	≥ 0 (CHECK)	Figé à la commande
reduction	DECIMAL(10,2)	≥ 0	10 % si applicable
prix_livraison	DECIMAL(8,2)	≥ 0	5 € + 0,59 €/km
prix_total	DECIMAL(10,2)	≥ 0	Prix TTC final
statut	VARCHAR(30)	CHECK (8 valeurs)	Cycle de vie
pret_materiel	BOOLEAN	DEFAULT FALSE	Vaisselle prêtée

restitution_materiel	BOOLEAN	DEFAULT FALSE	Matériel rendu
motif_annulation	TEXT	NULL	Obligatoire si annulée par employé
mode_contact_annulation	VARCHAR(10)	gsm mail (CHECK)	Mode utilisé
date_commande	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	—

Table avis

Colonne	Type	Contraintes	Rôle
avis_id	SERIAL	PK	Identifiant
utilisateur_id	INTEGER	FK, ON DELETE CASCADE	Auteur (relation <i>publie</i>)
commande_id	INTEGER	FK, UNIQUE	1 avis / commande
note	SMALLINT	BETWEEN 1 AND 5	Étoiles
commentaire	TEXT	10-2000 chars	Texte libre
statut	VARCHAR(15)	en_attente valide refuse	Modération
date_creation	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Dépôt
date_moderation	TIMESTAMP	NULL	Instant de validation/refus
moderateur_id	INTEGER	FK NULL, ON DELETE SET NULL	Employé/admin

Table horaire

Colonne	Type	Contraintes	Rôle
horaire_id	SERIAL	PK	Identifiant
jour	VARCHAR(15)	UNIQUE, CHECK (7 valeurs)	Lundi → Dimanche
ordre_jour	SMALLINT	UNIQUE, BETWEEN 1 AND 7	Tri
heure_ouverture	TIME	NULL si fermé	—
heure_fermeture	TIME	NULL si fermé	—
ferme	BOOLEAN	DEFAULT FALSE	Jour de fermeture

3.4 Modélisation NoSQL (MongoDB)

Une unique collection `statistiques_commandes` stocke une projection dénormalisée des commandes, adaptée aux agrégations rapides du dashboard admin. Structure d'un document :

```

{
  _id          : ObjectId(...),
  commande_id  : 42,                // rapprochement avec PostgreSQL
  numero_commande : "VG-2025-0042",
  menu_id      : 1,
  menu_titre   : "Menu de Noël Prestige",
  utilisateur_id : 123,
  statut       : "terminee",
  prix_menu    : 546.00,
  reduction    : 54.60,
  prix_livraison : 7.36,
  prix_total   : 498.76,
  nombre_personnes: 13,
  ville_livraison : "Talence",
  date_commande : ISODate("2025-11-20T14:30:00Z"),
  date_prestation : ISODate("2025-12-24T12:00:00Z")
}

```

Quatre index optimisent les agrégations les plus fréquentes :

Index	Rôle
<code>{ commande_id: 1 } unique</code>	Rapprochement clé PostgreSQL, empêche les doublons
<code>{ menu_titre: 1 }</code>	Group by « commandes par menu »
<code>{ statut: 1 }</code>	Filtrage rapide (exclut annulées, en_attente)
<code>{ date_commande: -1, statut: 1 }</code>	Composite pour CA temporel

4. Diagrammes UML

4.1 Diagramme de cas d'utilisation

Le diagramme suivant présente les cas d'utilisation groupés par acteur. L'administrateur hérite de l'ensemble des cas de l'employé (via `role_hierarchy` Symfony) plus deux cas exclusifs : la création d'employé et la consultation des statistiques.

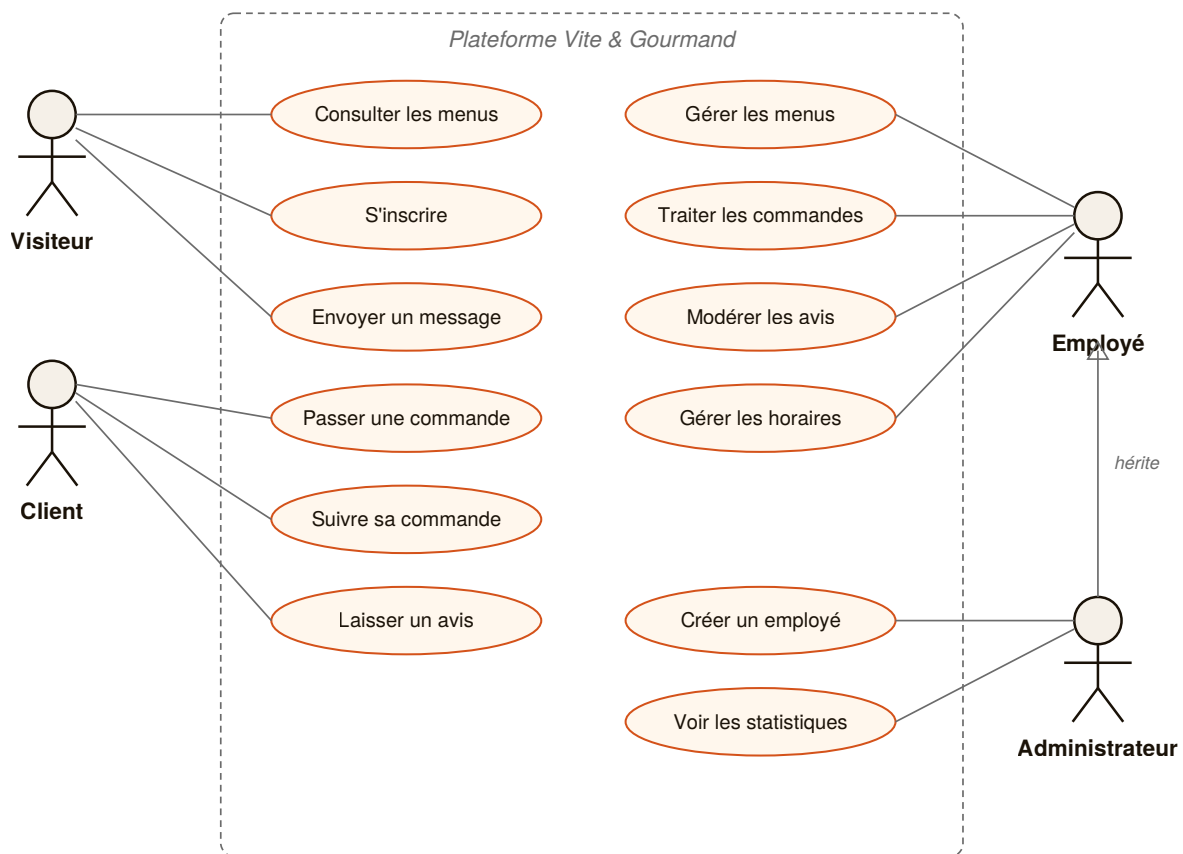


Diagramme UML de cas d'utilisation — 4 acteurs, 12 cas principaux

4.2 Diagramme de séquence — Création de commande

Le scénario suivant décrit la création d'une commande par un client authentifié. C'est le cas d'usage le plus riche : il implique une transaction Doctrine, un verrou pessimiste sur le menu (protection contre la course sur le stock), la synchronisation MongoDB automatique, et l'envoi de plusieurs e-mails transactionnels — le tout avec gestion d'erreurs.

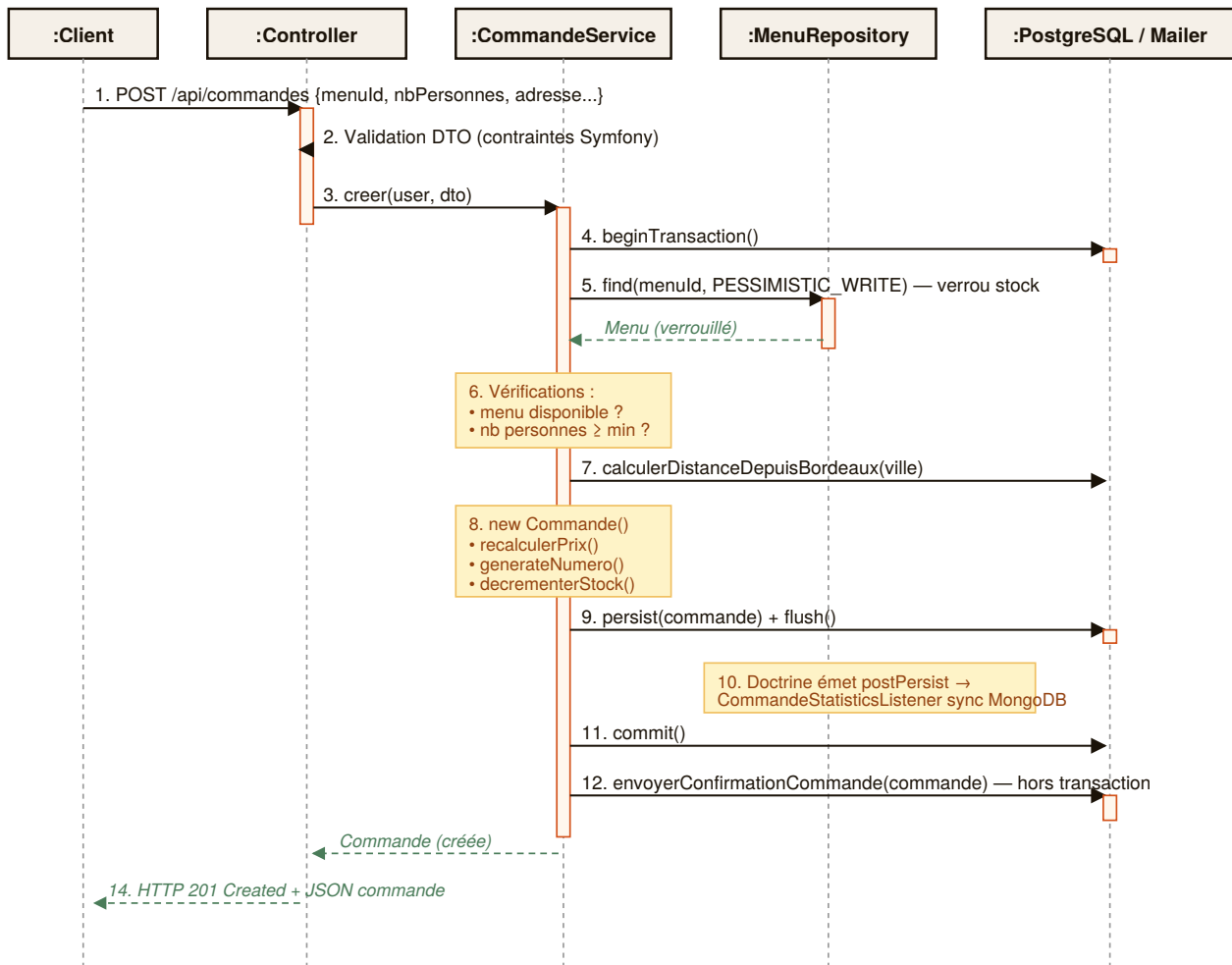


Diagramme de séquence — Création d'une commande (POST /api/commandes)

POINTS D'ATTENTION DU SCÉNARIO

1. Le verrou pessimiste (étape 5) empêche que deux clients simultanés ne consomment le même dernier stock — l'un des deux verra sa transaction rollback.
2. Le listener de synchronisation MongoDB (étape 10) est déclenché automatiquement par Doctrine sans coupler la logique métier au NoSQL.
3. L'envoi de l'e-mail (étape 12) a lieu *après* le commit — un échec SMTP n'annulera pas la commande.

5. API REST

5.1 Conventions générales

- **Format** : JSON uniquement (Content-Type: `application/json`)
- **Nommage** : snake_case pour les clés JSON, kebab-case pour les URLs
- **Codes HTTP** : usage sémantique strict (201 Created, 401 Unauthorized, 409 Conflict...)
- **Erreurs** : structure uniforme `{ "erreur": "message", "details": {...} }`
- **Authentification** : session PHP + cookie `PHPSESSID` HttpOnly, SameSite=Lax
- **CORS** : origine `FRONTEND_URL` uniquement, `credentials: true`

5.2 Liste complète des endpoints

Authentification (7 endpoints)

Méthode	Route	Rôle requis	Description
POST	/api/auth/inscription	Public	Créer un compte client
POST	/api/auth/login	Public	Se connecter (rate limit 5/min)
POST	/api/auth/logout	User	Se déconnecter
GET	/api/auth/me	User	Récupérer le profil connecté
POST	/api/auth/mot-de-passe-oublie	Public	Demande de réinitialisation
POST	/api/auth/reinitialiser	Public	Nouveau mot de passe avec token

Menus, plats, référentiels (12 endpoints)

Méthode	Route	Rôle requis	Description
GET	/api/menus?filtres...	Public	Liste filtrée
GET	/api/menus/{id}	Public	Détail d'un menu
GET	/api/menus/{id}/similaires	Public	Suggestions
POST	/api/employe/menus	Employé	Créer un menu
PUT	/api/employe/menus/{id}	Employé	Modifier
DEL	/api/employe/menus/{id}	Employé	Désactiver (soft delete)
GET	/api/employe/plats	Employé	Liste des plats
POST	/api/employe/plats	Employé	Créer un plat
PUT	/api/employe/plats/{id}	Employé	Modifier
DEL	/api/employe/plats/{id}	Employé	Supprimer

GET	/api/referentiels/themes	Public	Liste des thèmes
GET	/api/referentiels/regimes	Public	Liste des régimes
GET	/api/referentiels/allergenes	Public	Liste des allergènes

Commandes (10 endpoints)

Méthode	Route	Rôle requis	Description
POST	/api/commandes	User	Passer commande
GET	/api/commandes/mes-commandes	User	Mes commandes
GET	/api/commandes/{numero}	User	Détail (auteur uniquement)
PUT	/api/commandes/{numero}/ modifier	User	Modifier (si en attente)
DEL	/api/commandes/{numero}	User	Annuler (si en attente)
GET	/api/employe/commandes	Employé	Toutes commandes (filtres)
GET	/api/employe/commandes/{numero}	Employé	Détail complet
PUT	/api/employe/commandes/{numero}/ statut	Employé	Transitionner statut
DEL	/api/employe/commandes/{numero}	Employé	Annuler (motif obligatoire)
+ 3 alias sous /api/admin/commandes/...			

Avis (9 endpoints)

Méthode	Route	Rôle requis	Description
GET	/api/avis?limite=6	Public	Avis validés (accueil, RGPD)
POST	/api/avis	User	Déposer un avis
GET	/api/avis/mes-avis	User	Mes avis déposés
GET	/api/employe/avis?statut=...	Employé	File de modération
PUT	/api/employe/avis/{id}/valider	Employé	Publier
DEL	/api/employe/avis/{id}/refuser	Employé	Refuser
+ 3 alias sous /api/admin/avis/...			

Admin (7 endpoints)

Méthode	Route	Rôle requis	Description
GET	/api/admin/employes	Admin	Liste des employés
POST	/api/admin/employes	Admin	Créer un compte employé

PUT	/api/admin/employes/{id}/toggle-actif	Admin	Activer/désactiver
GET	/api/admin/stats/commandes-par-menu	Admin	Agrégation MongoDB (fallback SQL)
GET	/api/admin/stats/chiffre-affaires	Admin	CA par menu, filtrable par période

Divers (4 endpoints)

Méthode	Route	Rôle requis	Description
POST	/api/contact	Public	Formulaire de contact (rate limit 5/15min)
GET	/api/horaires	Public	Horaires d'ouverture
PUT	/api/employe/horaires	Employé	Mise à jour atomique 7 jours

5.3 Codes de réponse HTTP

Code	Signification	Cas d'usage
200	OK	Lecture réussie, modification réussie
201	Created	Création réussie (inscription, commande, avis...)
400	Bad Request	JSON malformé, paramètre invalide
401	Unauthorized	Non authentifié, session expirée
403	Forbidden	Rôle insuffisant, ressource d'un autre utilisateur
404	Not Found	Ressource inexistante
409	Conflict	Transition non autorisée, avis déjà déposé, e-mail existe
422	Unprocessable Entity	Erreurs de validation DTO, règle métier violée
429	Too Many Requests	Rate limit dépassé
500	Internal Server Error	Exception non gérée (loggée)

6. Sécurité

6.1 Authentification et session

Hashage des mots de passe

Les mots de passe sont hashés avec l'algorithme **Argon2id**, recommandé par l'ANSSI et l'OWASP. Ce choix est configuré dans `security.yaml` et appliqué transparentement par Symfony via l'interface `UserPasswordHasherInterface`. Aucun mot de passe n'est jamais stocké ni transmis en clair.

Politique de mots de passe

À l'inscription et à la réinitialisation, cinq règles sont appliquées côté serveur (les règles côté client via la jauge de force ne sont qu'une aide UX) :

- Longueur minimale de 10 caractères
- Au moins une majuscule
- Au moins une minuscule
- Au moins un chiffre
- Au moins un caractère spécial

Session et cookies

L'authentification s'appuie sur les sessions PHP natives, avec cookies `HttpOnly` (inaccessibles au JavaScript, protection XSS) et `SameSite=Lax` (protection CSRF pour les navigations cross-site). En production, le flag `Secure` impose HTTPS.

Réinitialisation de mot de passe

Le token de réinitialisation est un secret cryptographiquement fort (32 octets aléatoires), transmis dans l'URL de l'e-mail, mais stocké en base sous forme de **hash SHA-256** avec une expiration de 60 minutes. Ainsi une compromission de la base ne permet pas de rejouer les liens. La réponse à la demande est identique que l'e-mail existe ou non (protection contre l'énumération d'utilisateurs).

6.2 Contrôle d'accès (RBAC)

Trois rôles Symfony hiérarchisés sont définis via `role_hierarchy` :

```
role_hierarchy:
  ROLE_ADMINISTRATEUR: ROLE_EMPLOYE
  ROLE_EMPLOYE:        ROLE_UTILISATEUR
```

Ainsi l'administrateur hérite automatiquement des droits de l'employé, qui hérite des droits du client. Les contrôles sont appliqués à deux niveaux :

- **Access control** global défini dans `security.yaml` par pattern d'URL (par exemple `^/api/admin` exige `ROLE_ADMINISTRATEUR`)

- **Attribut** `#[IsGranted(...)]` au niveau des contrôleurs pour un contrôle fin, avec vérification supplémentaire dans les services lorsque le contexte métier l'exige (par exemple : « un client ne peut voir *que* ses propres commandes »)

6.3 Rate limiting et anti-abus

Endpoint	Limite	Fenêtre	Justification
<code>/api/auth/login</code>	5 tentatives	1 minute	Anti brute-force
<code>/api/auth/mot-de-passe-oublie</code>	3 demandes	15 minutes	Anti-spam SMTP
<code>/api/contact</code>	5 envois	15 minutes	Anti-flood contact

Ces limiteurs sont configurés dans `framework.yaml` et clefés par IP client. Un dépassement renvoie une réponse `429 Too Many Requests` claire. Le formulaire de contact intègre également un champ caché *honeypot* côté front pour piéger les bots.

6.4 Conformité RGPD

- **Minimisation** : seules les données strictement nécessaires sont collectées (prénom, nom, email, téléphone, adresse). Les avis publiés n'exposent que le prénom + initiale du nom.
- **Chiffrement** : mots de passe hashés (Argon2id), tokens hashés (SHA-256). En production, HTTPS est obligatoire (`Secure` cookies).
- **Portabilité** : l'utilisateur peut consulter ses commandes et avis via son espace personnel. Un export au format JSON peut être fourni sur demande.
- **Droit à l'oubli** : les comptes utilisateurs sont désactivables. Sur demande formelle, un script CLI peut anonymiser les données personnelles tout en préservant l'intégrité comptable des commandes.
- **Rétention** : la durée de conservation des tokens de réinitialisation est limitée à 60 minutes ; les sessions PHP expirent après inactivité.

7. Règles métier

7.1 Calcul des prix et réduction

La méthode `Commande::recalculerPrix()` applique la formule suivante :

```
prix_menu      = menu.prix_par_personne × nombre_personnes
reduction      = prix_menu × 0.10    si nombre_personnes ≥ minimum + 5
                = 0                  sinon
prix_livraison = 0                    si ville == "Bordeaux"
                = 5.00 + 0.59 × km    sinon
prix_total     = prix_menu - reduction + prix_livraison
```

PRÉCISION DÉCIMALE

Tous les montants sont stockés en type `DECIMAL` PostgreSQL (pas de `FLOAT`). Les calculs PHP passent par `number_format($v, 2, '.', '')` pour garantir la stabilité à deux décimales. Cette précaution évite les erreurs d'arrondi qui s'accumuleraient au fil des transactions.

7.2 Cycle de vie des commandes

Huit statuts sont définis dans l'énumération `StatutCommande`. Les transitions autorisées sont strictement contrôlées par la méthode `transitionsAutorisees()` :

```
en_attente      → accepte | annulee
accepte         → en_preparation | annulee
en_preparation  → en_cours_livraison | annulee
en_cours_livraison → livre
livre           → attente_materiel | terminee
attente_materiel → terminee
terminee, annulee → (statuts finaux, aucune transition)
```

Cette contrainte est vérifiée en base par `CHECK (statut IN (...))` et par le service `CommandeService::transitionnerStatut()`. Une tentative de transition interdite renvoie `409 Conflict`.

7.3 Modération des avis

Les avis suivent un flux à trois états : `en_attente` → `valide` ou `refuse`. Seuls les avis validés sont exposés publiquement. Trois règles métier protègent la qualité et l'unicité :

1. **Commande terminée obligatoire** — un client ne peut déposer un avis que sur une commande dont le statut est `terminee`, vérifié via `StatutCommande::autoriseDepotAvis()`.
2. **Un seul avis par commande** — la contrainte `UNIQUE (commande_id)` en base est le garde-fou définitif. Le service pré-vérifie côté PHP pour retourner un message d'erreur clair (409 Conflict).
3. **Traçabilité** — le modérateur (`moderateur_id`) et la date de décision (`date_moderation`) sont conservés en base pour audit.

DOCUMENTATION COMPLÉMENTAIRE

La documentation utilisateur (guide d'utilisation par profil) est fournie en document séparé : *Vite & Gourmand — Manuel utilisateur*. La charte graphique et la documentation de déploiement font également l'objet de livrables dédiés.